JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class TemporalQueries

java.lang.Object
    java.time.temporal.TemporalQueries

```
public final class TemporalQueries
extends Object
```

Common implementations of `TemporalQuery`.

This class provides common implementations of `TemporalQuery`. These are defined here as they must be constants, and the definition of lambdas does not guarantee that. By assigning them once here, they become 'normal' Java constants.

Queries are a key tool for extracting information from temporal objects. They exist to externalize the process of querying, permitting different approaches, as per the strategy design pattern. Examples might be a query that checks if the date is the day before February 29th in a leap year, or calculates the number of days to your next birthday.

The `TemporalField` interface provides another mechanism for querying temporal objects. That interface is limited to returning a `long`. By contrast, queries can return any type.

There are two equivalent ways of using a `TemporalQuery`. The first is to invoke the method on this interface directly. The second is to use `TemporalAccessor.query(TemporalQuery)`:

```
  // these two lines are equivalent, but the second approach is recommended
  result = thisQuery.queryFrom(temporal);
  result = temporal.query(thisQuery);
```

It is recommended to use the second approach, `query(thisQuery)`, as it is a lot clearer to read in code.

The most common implementations are method references, such as `LocalDate::from` and `ZoneId::from`. Additional common queries are provided to return:

- a Chronology,
- a LocalDate,
- a LocalTime,
- a ZoneOffset,
- a precision,
- a zone, or
- a zoneId.

Since:

1.8

Method Summary

All Methods	Static Methods	

Modifier and Type	Method	Description
static TemporalQuery<Chronology>	**chronology**()	A query for the Chronology.
static TemporalQuery<LocalDate>	**localDate**()	A query for LocalDate returning null if not found.
static TemporalQuery<LocalTime>	**localTime**()	A query for LocalTime returning null if not found.
static TemporalQuery<ZoneOffset>	**offset**()	A query for ZoneOffset returning null if not found.
static TemporalQuery<TemporalUnit>	**precision**()	A query for the smallest supported unit.
static TemporalQuery<ZoneId>	**zone**()	A lenient query for the ZoneId, falling back to the ZoneOffset.
static TemporalQuery<ZoneId>	**zoneId**()	A strict query for the ZoneId.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Method Details

zoneId

```
public static TemporalQuery<ZoneId> zoneId()
```

A strict query for the `ZoneId`.

This queries a `TemporalAccessor` for the zone. The zone is only returned if the date-time conceptually contains a `ZoneId`. It will not be returned if the date-time only conceptually has an `ZoneOffset`. Thus a `ZonedDateTime` will return the result of `getZone()`, but an `OffsetDateTime` will return null.

In most cases, applications should use `zone()` as this query is too strict.

The result from JDK classes implementing `TemporalAccessor` is as follows:
`LocalDate` returns null
`LocalTime` returns null
`LocalDateTime` returns null
`ZonedDateTime` returns the associated zone
`OffsetTime` returns null
`OffsetDateTime` returns null
`ChronoLocalDate` returns null
`ChronoLocalDateTime` returns null
`ChronoZonedDateTime` returns the associated zone
`Era` returns null
`DayOfWeek` returns null
`Month` returns null
`Year` returns null
`YearMonth` returns null
`MonthDay` returns null
`ZoneOffset` returns null
`Instant` returns null

Returns:

a query that can obtain the zone ID of a temporal, not null

chronology

```
public static TemporalQuery<Chronology> chronology()
```

A query for the `Chronology`.

This queries a `TemporalAccessor` for the chronology. If the target `TemporalAccessor` represents a date, or part of a date, then it should return the chronology that the date is expressed in. As a result of this definition, objects only representing time, such as `LocalTime`, will return null.

The result from JDK classes implementing `TemporalAccessor` is as follows:
`LocalDate` returns `IsoChronology.INSTANCE`
`LocalTime` returns null (does not represent a date)
`LocalDateTime` returns `IsoChronology.INSTANCE`
`ZonedDateTime` returns `IsoChronology.INSTANCE`

`OffsetTime` returns null (does not represent a date)
`OffsetDateTime` returns `IsoChronology.INSTANCE`
`ChronoLocalDate` returns the associated chronology
`ChronoLocalDateTime` returns the associated chronology
`ChronoZonedDateTime` returns the associated chronology
`Era` returns the associated chronology
`DayOfWeek` returns null (shared across chronologies)
`Month` returns `IsoChronology.INSTANCE`
`Year` returns `IsoChronology.INSTANCE`
`YearMonth` returns `IsoChronology.INSTANCE`
`MonthDay` returns null `IsoChronology.INSTANCE`
`ZoneOffset` returns null (does not represent a date)
`Instant` returns null (does not represent a date)

The method `Chronology.from(TemporalAccessor)` can be used as a `TemporalQuery` via a method reference, `Chronology::from`. That method is equivalent to this query, except that it throws an exception if a chronology cannot be obtained.

Returns:

a query that can obtain the chronology of a temporal, not null

precision

```
public static TemporalQuery<TemporalUnit> precision()
```

A query for the smallest supported unit.

This queries a `TemporalAccessor` for the time precision. If the target `TemporalAccessor` represents a consistent or complete date-time, date or time then this must return the smallest precision actually supported. Note that fields such as `NANO_OF_DAY` and `NANO_OF_SECOND` are defined to always return ignoring the precision, thus this is the only way to find the actual smallest supported unit. For example, were `GregorianCalendar` to implement `TemporalAccessor` it would return a precision of `MILLIS`.

The result from JDK classes implementing `TemporalAccessor` is as follows:
`LocalDate` returns `DAYS`
`LocalTime` returns `NANOS`
`LocalDateTime` returns `NANOS`
`ZonedDateTime` returns `NANOS`
`OffsetTime` returns `NANOS`
`OffsetDateTime` returns `NANOS`
`ChronoLocalDate` returns `DAYS`
`ChronoLocalDateTime` returns `NANOS`
`ChronoZonedDateTime` returns `NANOS`

Era returns ERAS
DayOfWeek returns DAYS
Month returns MONTHS
Year returns YEARS
YearMonth returns MONTHS
MonthDay returns null (does not represent a complete date or time)
ZoneOffset returns null (does not represent a date or time)
Instant returns NANOS

Returns:

a query that can obtain the precision of a temporal, not null

zone

```
public static TemporalQuery<ZoneId> zone()
```

A lenient query for the ZoneId, falling back to the ZoneOffset.

This queries a TemporalAccessor for the zone. It first tries to obtain the zone, using zoneId(). If that is not found it tries to obtain the offset(). Thus a ZonedDateTime will return the result of getZone(), while an OffsetDateTime will return the result of getOffset().

In most cases, applications should use this query rather than #zoneId().

The method ZoneId.from(TemporalAccessor) can be used as a TemporalQuery via a method reference, ZoneId::from. That method is equivalent to this query, except that it throws an exception if a zone cannot be obtained.

Returns:

a query that can obtain the zone ID or offset of a temporal, not null

offset

```
public static TemporalQuery<ZoneOffset> offset()
```

A query for ZoneOffset returning null if not found.

This returns a TemporalQuery that can be used to query a temporal object for the offset. The query will return null if the temporal object cannot supply an offset.

The query implementation examines the OFFSET_SECONDS field and uses it to create a ZoneOffset.

The method ZoneOffset.from(TemporalAccessor) can be used as a TemporalQuery via a method reference, ZoneOffset::from. This query and

`ZoneOffset::from` will return the same result if the temporal object contains an offset. If the temporal object does not contain an offset, then the method reference will throw an exception, whereas this query will return null.

Returns:

a query that can obtain the offset of a temporal, not null

localDate

```
public static TemporalQuery<LocalDate> localDate()
```

A query for `LocalDate` returning null if not found.

This returns a `TemporalQuery` that can be used to query a temporal object for the local date. The query will return null if the temporal object cannot supply a local date.

The query implementation examines the `EPOCH_DAY` field and uses it to create a `LocalDate`.

The method `LocalDate.from(TemporalAccessor)` can be used as a `TemporalQuery` via a method reference, `LocalDate::from`. This query and `LocalDate::from` will return the same result if the temporal object contains a date. If the temporal object does not contain a date, then the method reference will throw an exception, whereas this query will return null.

Returns:

a query that can obtain the date of a temporal, not null

localTime

```
public static TemporalQuery<LocalTime> localTime()
```

A query for `LocalTime` returning null if not found.

This returns a `TemporalQuery` that can be used to query a temporal object for the local time. The query will return null if the temporal object cannot supply a local time.

The query implementation examines the `NANO_OF_DAY` field and uses it to create a `LocalTime`.

The method `LocalTime.from(TemporalAccessor)` can be used as a `TemporalQuery` via a method reference, `LocalTime::from`. This query and `LocalTime::from` will return the same result if the temporal object contains a time. If the temporal object does not contain a time, then the method reference will throw an exception, whereas this query will return null.

Returns:

a query that can obtain the time of a temporal, not null

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.